

Q1. Given a char[] s, calculate no. of pairs $[i, j]$, such that $i < j$ & $s[i] == 'a'$, & $s[j] == 'g'$, all characters are lowercase alphabets $[a, b, c, d, \dots]$

ex $\Rightarrow$ arr $\Rightarrow$ b a a g d c a g
0 1 2 3 4 5 6 7

valid pairs $\leftarrow$ $\langle 1, 3 \rangle$ $\langle 2, 3 \rangle$ $\langle 6, 7 \rangle$
 $\langle 1, 7 \rangle$ $\langle 2, 7 \rangle$
 ~~$\langle 3, 6 \rangle$~~ ~~$\langle 6, 3 \rangle \rightarrow i < j$~~

$Q_p = 5$

ex $\Rightarrow$ arr $\Rightarrow$ b c a g g a a g
0 1 2 3 4 5 6 7

 $Q_p = 5$

ex $\Rightarrow$ arr $\Rightarrow$ a c g d g a g
0 1 2 3 4 5 6

 $Q_p = 4$

Brute force

check all possible pairs of
(i, j).

```
c = 0
for (i = 0; i < N; i++) {
    for (j = i + 1; j < N; j++) {
        if (s[i] == 'a' && s[j] == 'g')
            c++
    }
}
print(c)
```

TC $\Rightarrow O(N^2)$
SC $\Rightarrow O(1)$

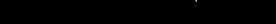
* if $s[i] \neq 'a'$, then second loop is not required.

```
for (i = 0; i < N; i++) {
    if (s[i] == 'a') {
        for (j = i + 1; j < N; j++) {
            if (s[j] == 'g')
                c++
        }
    }
}
print(c)
```

still

TC $\Rightarrow O(N^2)$
SC $\Rightarrow O(1)$

↓ ↓ ↓
a a a a a a a a



Handwritten diagram illustrating the selection sort algorithm on the array `adgaqagfg`. The array is indexed 0 to 8. The diagram shows three iterations of finding the minimum element:

- Iteration 1:** $i=0$. A vertical line at index 0 has an arrow pointing to index 3, where the element `a` is located. A horizontal arrow from this line points to the label `count(g)` with the value `4`.
- Iteration 2:** $i=3$. A vertical line at index 3 has an arrow pointing to index 5, where the element `q` is located. A horizontal arrow from this line points to the value `3`.
- Iteration 3:** $i=5$. A vertical line at index 5 has an arrow pointing to index 7, where the element `g` is located. A horizontal arrow from this line points to the value `2`.

$$4 + 3 + 2 = 9$$
$$a \text{-----} b$$
[illegible]

$$Q_p = 9$$

TC $\Rightarrow O(N)$
SC $\Rightarrow O(1)$

pseudo

ans = 0, cg = 0.

```
for (i = N-1; i >= 0; i--) {  
    if (arr[i] == 'g') {  
        cg++;  
    }  
    else if (arr[i] == 'a') {  
        ans = ans + cg;  
    }  
}  
print(ans)
```

TC $\Rightarrow O(N)$

SC $\Rightarrow O(1)$

idea 2

For every 'g' calculate no. of 'a' on the
(left - right) side :-

what to carry :-

1) ans

2) count of a (ca)

→ left to right

How $\rightarrow$ pseudocode

Q2. leaders in an array:

Given an $arr[N]$, you have to find all leaders in $arr[]$.
An element is a leader, if it is strictly greater than all its elements on the right side.

: Note: $[N-1]$ is always a leader.

ex $\Rightarrow arr[] \Rightarrow$ 15 -1 7 2 5 4 2 3
✓ ✗ ✓ ✗ ✓ ✓ ✗ ✓

$Qp = 5$.

$arr[] \Rightarrow$ 10 7 9 3 2 4 5
✓ ✗ ✓ ✗ ✗ ✗ ✓

$Qp = 3$.

$arr[] \Rightarrow$ 8 -2 4 7 6 5 1
✓ ✗ ✗ ✓ ✓ ✓ ✓

$Qp = 5$.

$arr[] \Rightarrow$ 10 8 8
✓ ✗ ✓

$Qp = 2$.

Brute force

take each element, and compare
to all elements forward right, and
check if its leader

$$TC \Rightarrow O(N^2) \quad SC \Rightarrow O(1)$$

pseudo $\rightarrow$ How

Optimised

10 7 9 3 2 4 5

$9 > 5 \rightarrow 9$ is a leader

$3 < 5 \rightarrow 3$ is not a leader.

for any idx i , need to know the max from $(i+1$ to $N-1)$

if (arr[i] > max)

C++

$\downarrow$						
10	7	9	3	2	4	5
0	1	2	3	4	5	6

count = 1 & 3 $\rightarrow$ OP = 3
 max = ~~5~~ & 10

pseudo

```
c = 1, max = arr[N-1]
for (i = N-2; i >= 0; i--) {
    if (arr[i] > max) {
        c++,
        max = arr[i]
    }
}
print(c)
```

TC $\Rightarrow O(N)$

SC $\Rightarrow O(1)$

$\Rightarrow$ Subarrays :

Basis $\rightarrow$

1) Continuous part of an array is called subarray

1) a single element is a subarray

ex $\Rightarrow$ arr = [1 4 3 2 5]

[4], [3], [2] - - subarrays

2) entire array is also a subarray

ex $\Rightarrow$ arr = [1 4 3]

$\uparrow$
also a subarray.

3) empty[] can't be a subarray.

ex) arr[9] $\Rightarrow$

-3	4	6	2	8	7	14	9	21
0	1	2	3	4	5	6	7	8

$\Rightarrow$ idx $\Rightarrow$ [2 3 4 5] $\checkmark$ $\Rightarrow$ [2-5]

$\Rightarrow$ idx $\Rightarrow$ [3 4 6 7 8] $\times \times \rightarrow$ 6 is skipped

$\Rightarrow$ idx $\Rightarrow$ [1-3] $\checkmark$

$\Rightarrow$ idx $\Rightarrow$ [4] $\checkmark$

$\Rightarrow$ idx $\Rightarrow$ [5-5] $\checkmark$

[1] $\rightarrow \times \times \times$.

subarray $\Rightarrow$ [3-7] $\Rightarrow$

3	4	5	6	7
$\underbrace{\hspace{1.5cm}}$				

 $\rightarrow$ 5 elements
 $\downarrow$
 $\underbrace{7-3+1}_{5} \Rightarrow$ elements.

subarray $\Rightarrow$ [s-e] $\Rightarrow$ length = $e-s+1$
 $\underbrace{\hspace{10cm}}$

⇒ usage of - predefined func:-

⇒ $\min(a, b) \rightarrow$ returns the $\min$ of a & b .
TC $\Rightarrow O(1)$

⇒ $\max(a, b) \rightarrow$ returns the $\max$ of a & b .
TC $\Rightarrow O(1)$

⇒ $\text{sort}(arr) \rightarrow$ arranges the array into asc order
by default;
TC $\Rightarrow O(N \log N)$

⇒ always consider TC of predefined func in
overall TC.
Σ

→ find the $\min$ no. of an array:-

$\min^u = arr[0]$

for ($i=0$; $i < N$; $i++$) {

$\min^u = \min(arr[i], \min^u)$

}

}

```
if (arr[i] < min) {  
    min = arr[i];  
}
```

Q 3. Closest Min Max

Given an array find the length of smallest subarray which contains both min & max of array?

ex $\Rightarrow$ arr $\Rightarrow$ 1 2 3 1 3 4 6 4 6 3
 0 1 2 3 4 5 6 7 8 9

min = 1, max = 6.

idx $\Rightarrow$ [0 8] = len
 9.

[3 8] = 6.

[3 6] = 4 $\rightarrow$ shortest $\Rightarrow$ dp = 4

ex $\Rightarrow$ arr $\Rightarrow$ 2 2 6 4 5 1 5 2 6 4 1
 0 1 2 3 4 5 6 7 8 9 10

min = 1

max = 6

idx [1 7] $\Rightarrow$ len
 7

[2 5] $\Rightarrow$ 4

[8 10] $\Rightarrow$ ③ $\rightarrow$ dp = 3

ex $\Rightarrow$ arr[] = 8 8 8 8 8 8 8

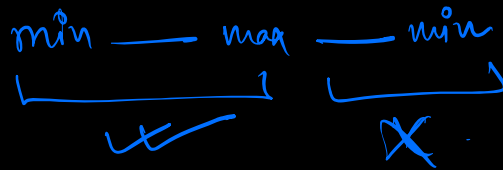
min = 8

max = 8

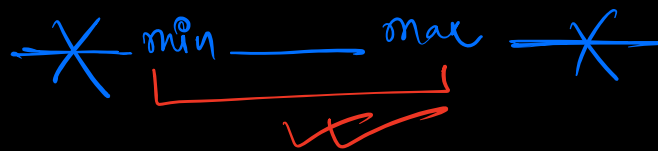
dp = 1.

Observations

- i) our final shortest subarray, can only contain
1 min & 1 max



- ii) min & max should always be at boundaries.



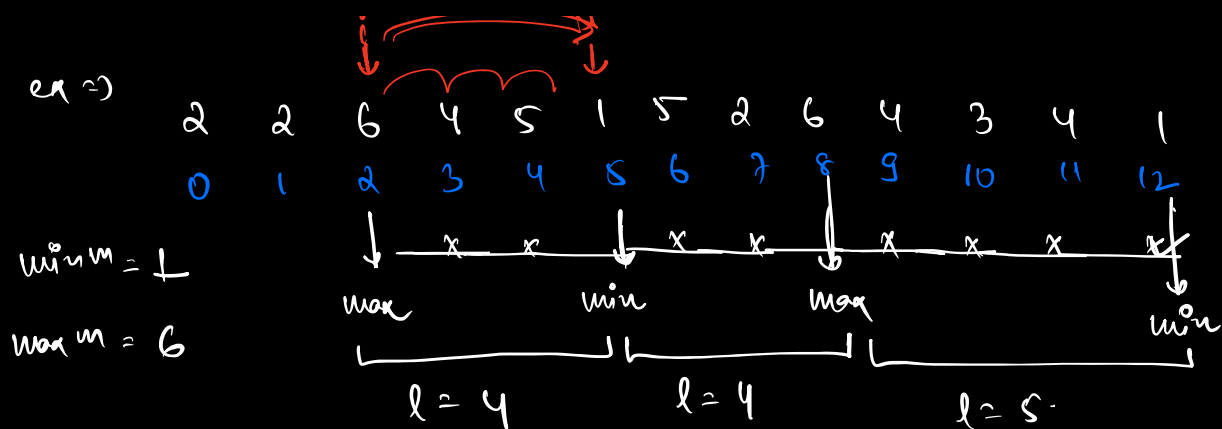
- iii) from pt 1 & pt 2, there are only
2 possibilities

Case 1 → [min — max]

if standing at min, find the nearest max
on right side.

Case 2 → [max — min]

if standing at max, find the nearest min
on right side.



Qp $\Rightarrow$ $\min len = 4$

pseudo

ans = N.

iterate and get $\max m \rightarrow \max$

iterate and get $\min m \rightarrow \min$

for ($i = 0; i < N; i++$) {

if ($arr[i] == \min$) { // find nearest max

for ($j = i+1; j < N; j++$) {

if ($arr[j] == \max$) {

ans = $\min(ans, j - i + 1)$.

break

}

}

}

else if ($arr[i] == \max$) { // find nearest min

for ($j = i+1; j < N; j++$) {

if ($arr[j] == \min$) {

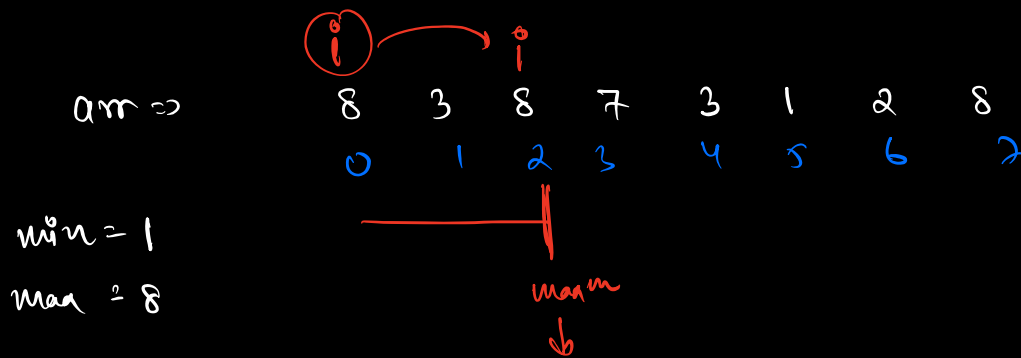
ans = $\min(ans, j - i + 1)$.

```

    }
    }
    } break

```

$TC \Rightarrow O(N^2)$
 $SC \Rightarrow O(1)$



$break \leftarrow i$

break and start again to find min, $i = 2$

update break/for pseudo code with this code

- $\Rightarrow$ if starting from max to find min.
and we encounter a max $\rightarrow$ break & start again from new max
- $\Rightarrow$ if starting from min to find max
and we encounter a min $\rightarrow$ break & start again from new min

$TC \Rightarrow O(N^2) \rightarrow$ Now $TC \Rightarrow O(N)$

$\rightarrow$ first next class

min = 1, max = 6.

Ans = N
min = -1
max = -1

ex $\Rightarrow$ $\downarrow$

	0	1	2	3	4	5	6	7	8	9	10	11	12
	1	6	4	6	5	1	5	2	6	4	4	2	1
			X		X		X	X		X	X	X	
min = 0													min = 12
max = 1		max = 1		max = 3		min = 5			min = 12				max = -1
		min = 5		min = 5		max = 8			max = 8				
l = 2		l = 5		l = 3		l = 4, ans = 4			l = 5, ans = 5				
ans = 2		ans = 3		ans = 3									

$\rightarrow$ Q1 = 2

TC $\Rightarrow O(N)$
SC $\Rightarrow O(1)$

pseudo

1) iterate $\rightarrow$ get min & max
if (min == max)
return 1

11) ans = N
min = -1
max = -1

for (i = N-1; i >= 0; i--) {
if (arr[i] == min) {

if (min != -1
& max != -1

min = i $\rightarrow$ abs.
l = (max - min) + 1

```

        ans = min(ans, l)
    }
    else if (arr[i] == max) {
        max = i
        l = (max - min) + 1
        ans = min(ans, l)
    }
}
print(ans)

```

*if (min != -1
 & max != -1*

also.

```

max = arr[0], c = 0.
for (i = 0; i < N; i++) {
    if (arr[i] == max)
        c++;
    else if (arr[i] < max) {
        max = arr[i]
        c = 1
    }
}
}

```